Methodology

The DSL is implemented as eight cooperating modules under `src/template_methods_paper/methods_dsl/`, each corresponding to one stage of a BPL-inspired pipeline (Bota Biosciences 2026). This section walks the pipeline stage by stage, naming the function or class that implements each design decision so every claim below is directly checkable against `src/template_methods_paper/methods_dsl/`.

Controlled vocabulary (`vocabulary.py`) I

A `StepKind` is one of 9 controlled intents — `TRANSFER`, `ADD`, `MIX`, `INCUBATE`, `MEASURE`, `WAIT`, `COMPUTE`, `VALIDATE`, `ANNOTATE` — and a `Target` is one of 3 execution backends — `HUMAN`, `AUTOMATED`, `SIMULATION`. `target_accepts` encodes which kinds require an automated backend: only `COMPUTE` has no manual equivalent in this DSL's scope, so `HUMAN` and `SIMULATION` accept every other kind. This is the domain-neutral generalization of BPL's protocol-level verbs (`add_reagent`, `transfer`, `incubate`): a step names *what* happens, never *how* a particular backend performs it.

Dimensional safety (`units.py`) I

Every `Quantity(value, unit)` resolves its `unit` to one of eight `Dimension` members (mass, volume, temperature, time, molar concentration, mass concentration, count, dimensionless) via `dimension_of`, drawing from a controlled table of 18 unit strings. Molar concentration (`mol/L`, `mM`) and mass concentration (`g/L`) are separate dimensions rather than one shared "concentration" dimension, since converting between them needs a substance's molar mass that this DSL does not carry. `check_compatible` raises `DimensionError` the moment two quantities with different dimensions are combined — the concrete realization of BPL's design principle that "the type system catches `mL + g` at compile time, not at the bench." Temperature is tracked as its own dimension with no shared base unit (`degC` and

Dimensional safety (`units.py`) I

`K` are never auto-converted), since this DSL has no use for that conversion and an incorrect affine conversion is worse than refusing one.

Method model (`model.py`) I

A `Method` is a name, a version, a target, a tuple of `Resource` declarations (anything a step reads from or writes to — generalizing BPL's `reagent`/`labware`), a tuple of method-level `Parameters`, and a tuple of `Steps`. Each `Step` carries a `step_id`, a `StepKind`, a `Target`, its own parameters, an optional expected duration (must be a time `Quantity`), and a `depends_on` tuple of prerequisite `step_ids` — the explicit DAG edges this section's compilation stage resolves. All four dataclasses are frozen; `__post_init__` rejects malformed shapes immediately (empty names, self-dependency, a non-time `expected_duration`) so structurally invalid methods cannot even be constructed, collapsing BPL's syntax gate into Python's own construction-time checks.

Staged validation (`validation.py`) I

`run_all_gates` runs exactly 4 gates in fixed order, mirroring BPL's staged short-circuit (a syntax failure never reaches the plan gate):

Staged validation (`validation.py`) I

1. **`structural_gate`** — every `step_id` is unique and every `depends_on` entry resolves to a real step.
2. **`semantic_gate`** — every `Quantity` attached to a resource, a method parameter, a step's expected duration, or a step parameter resolves to a known `Dimension` (catches the unit-vocabulary violation a frozen dataclass's `__post_init__` cannot, since `Quantity` does not validate its unit eagerly).
3. **`plan_gate`** — the step-dependency graph is acyclic, checked by attempting `topological_order` and catching `CycleError`.
4. **`target_gate`** — every step's target is compatible with the method's target (`HUMAN` methods accept only `HUMAN` steps; `AUTOMATED` methods accept both; `SIMULATION` methods accept only `SIMULATION` steps) and every step's kind is executable on its assigned target.

Staged validation (`validation.py`) I

If either of the first two gates fails, `run_all_gates` returns early with only those two results — `plan_gate` and `target_gate` assume a structurally and semantically valid method and would otherwise report misleading secondary failures.

Deterministic compilation (compiler.py) I

compile_method first calls run_all_gates and raises MethodValidationError (carrying every failed gate's issues) if any gate fails. On success, topological_order schedules the validated steps with Kahn's algorithm (Kahn 1962): repeatedly remove a step whose dependencies are all already scheduled, breaking ties by ascending step_id so the same method always yields the same order — Python does not guarantee dict/set iteration order is stable for this purpose, so the tie-break is explicit, not incidental. The scheduled steps are then encoded as a canonical, sort-keys JSON payload and hashed with SHA-256 (_compute_plan_hash) into Plan.plan_hash. Because the hash is computed purely from method_name, method_version, target, and

Deterministic compilation (`compiler.py`) I

each step's `step_id`/`name`/`kind`/`target`/scheduled `order` — never from a wall-clock timestamp or a UUID — recompiling the same `Method` object always produces the same `plan_hash`, which sec. 4 verifies live rather than asserts.

Export (`export.py`) I

A compiled `Plan` renders to four formats, mirroring BPL's "CSV/XLSX worklists, workflow graphs" export surface at a scope appropriate for a template exemplar: `to_worklist_markdown` (a numbered, human-readable worklist), `to_csv_rows`/`write_csv` (machine-readable rows), `to_mermaid` (a `flowchart TD` showing scheduled order), and `to_json`/`write_json` (the exact canonical JSON the plan hash was computed over, so a reader can independently verify `Plan.plan_hash` by re-hashing the exported file).

Provenance (`trust.py`) I

`ProvenanceTier` orders three levels of trust — `DECLARED`, `CALIBRATED`, `VERIFIED` — generalizing BPL's audit model. `append_record` extends an immutable hash-chain of `StateRecords`, each hashing its own `key`/`value`/`tier`/`prev_hash` (Merkle 1987); `verify_chain` recomputes every record's hash and checks it against the recorded `prev_hash` chain. This is a consistency check, not a cryptographic tamper-proof guarantee against an actor with write access to the whole stored chain: it detects in-chain tampering (any record after a tampered one no longer matches) but cannot detect a chain rewritten from record zero, exactly the boundary BPL's own "hash-chained" (not cryptographically signed) audit model claims.

Zero-mock testing methodology I

The project is governed by a strict zero-mock policy, evaluated by running `uv run pytest projects/templates/template_methods_paper/tests` during the build.

Zero-mock testing methodology I

1. **Library tests** exercise every gate, the compiler, the exporters, and the trust module against real `Method` objects — `conftest.py` ships one fixture per gate-failure mode (unknown dependency, duplicate step id, unknown unit, cyclic dependency, target mismatch) plus a linear-chain and a diamond-DAG method for scheduling tests. No `unittest.mock`, no `MagicMock`, no `@patch`.
2. **Script test** runs `run_methods_analysis()` against a temporary output root and asserts that real worklist/CSV/Mermaid/JSON files, reports, and a real PNG figure are written.
3. **Determinism tests** compile the same `Method` object twice and assert `plan_hash` equality live, rather than asserting against a hardcoded hash literal — a literal would silently stop testing the moment the compiler's hash input changed.
4. **Coverage gate**: CI enforces a $\geq$90% statement-coverage gate on Falling below that floor fails the gate outright (`--cov-fail-under` enforces it). `projects/templates/template_methods_paper/src/`; the live figure is tracked in `docs/_generated/COUNTS.md`.